

CLOUD COMPUTING

LAB NO # 13



SUBMITTED TO:

Sir Shohaib

SUBMITTED BY:

Name: Amina Noor

Reg No. 2023-BSE-007

Semester: V-A

CLOUD COMPUTING

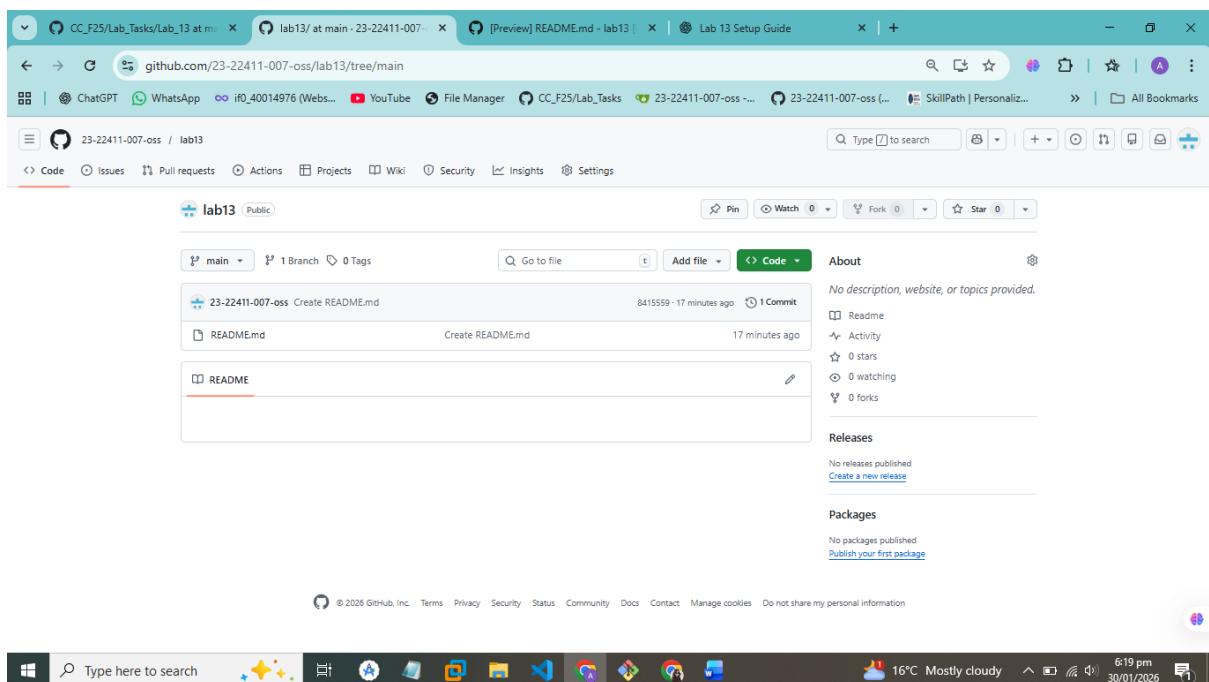
LAB NO # 13

Task 0 — Lab Setup (Codespace & GH CLI)

STEP 1: Create the GitHub Repository (via browser)

Before GH CLI can create a Codespace, the **repo must exist**.

1. Open **GitHub.com**
2. Click **New Repository**
3. Fill in:
 - **Repository name:** Lab13
 - **Owner:** CC_<YourName>_<YourRollNumber>
 - **Visibility:** **Public**
4. Click **Create repository**



STEP 2: Open Codespaces UI

Open this link directly:

👉 <https://github.com/codespaces>

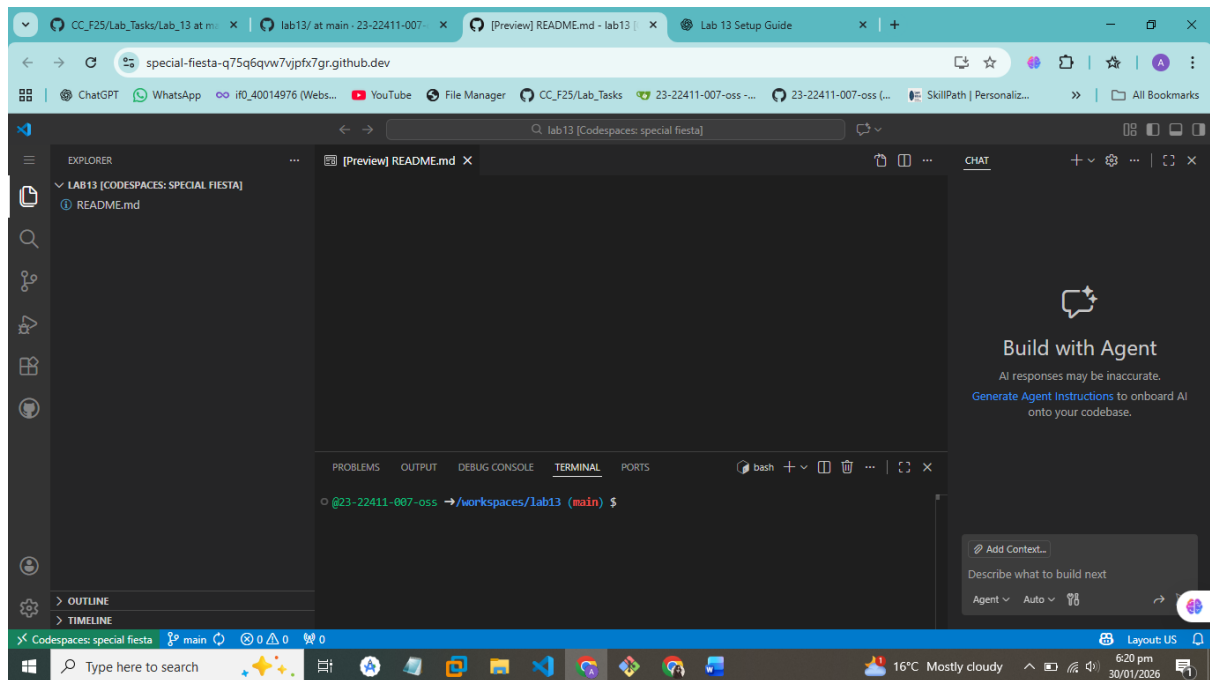
Click **New Codespace**

Fill:

- Repository: CC_<YourName>_<YourRollNumber>/Lab13
- Branch: main
- Region: default
- Machine type: default

Click **Create codespace**

⌚ Wait ~1–2 minutes → VS Code web opens.

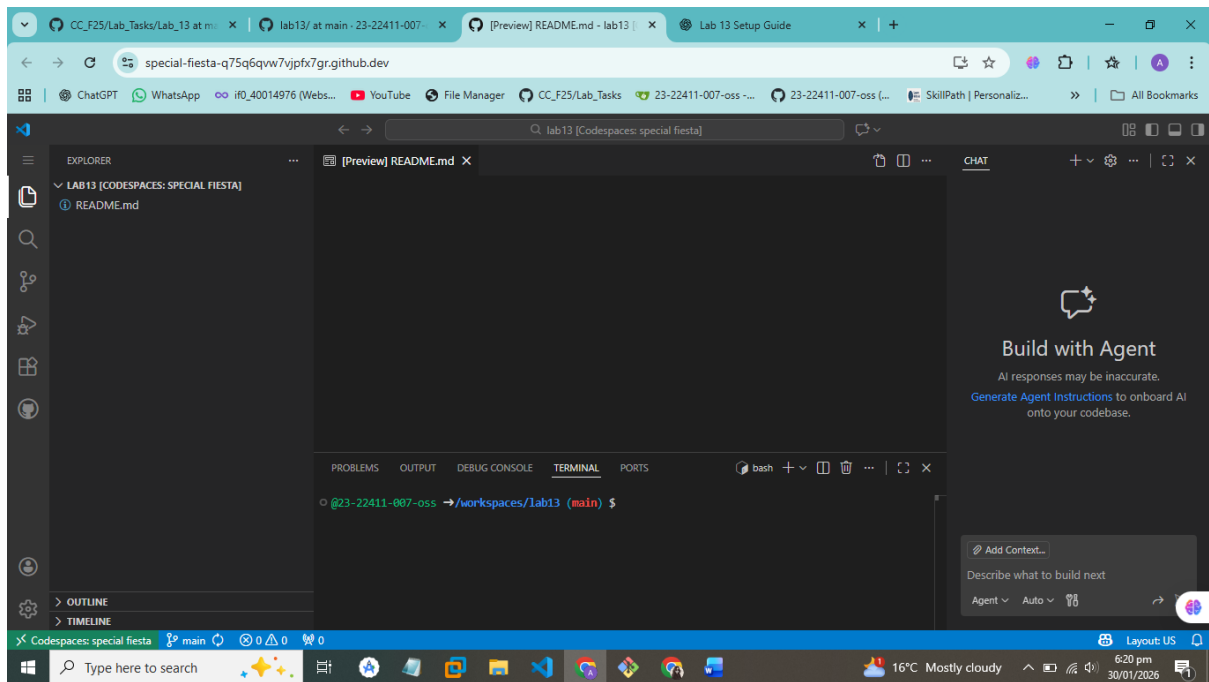


STEP 3: Open Terminal INSIDE Codespace

In the Codespace (VS Code web):

- Click **Terminal** → **New Terminal**

💡 From this point onward, **everything is inside Codespace** ✓



STEP 4: Authenticate GitHub CLI (inside Codespace)

Run:

```
gh auth login
```

Choose:

- ✓ GitHub.com
- ✓ HTTPS
- ✓ Login with a web browser
 - Copy the **one-time code**
 - Open the link shown
 - Paste the code
 - Authorize GitHub

Verify login:

```
gh auth status
```

You should see:

- ✓ Logged in to github.com

```
@23-22411-007-oss →/workspaces/lab13 (main) $ gh auth login
? Where do you use GitHub? GitHub.com
The value of the GITHUB_TOKEN environment variable is being used for authentication.
To have GitHub CLI store credentials instead, first clear the value from the environment.
@23-22411-007-oss →/workspaces/lab13 (main) $ gh auth status
github.com
✓ Logged in to github.com account 23-22411-007-oss (GITHUB_TOKEN)
- Active account: true
- Git operations protocol: https
- Token: ghu_*****
@23-22411-007-oss →/workspaces/lab13 (main) $
```

STEP 5: Create Repository using GH CLI (required by lab)

Now run:

```
gh repo create CC_<YourName>_<YourRollNumber>/Lab13 --public
```

STEP 6: Create Codespace via GH CLI

Run:

```
gh codespace create --repo CC_<YourName>_<YourRollNumber>/Lab13
```

Then list codespaces:

```
gh codespace list
```

Example output:

NAME	REPOSITORY	STATE
happy-mango-7	CC_Amina_23-22411-007/Lab13	Available

```
codespaces-4fcd84
@23-22411-007-oss →/workspaces/lab13 (main) $ gh codespace list
NAME          DISPLAY NAME  REPOSITORY      BRANCH  STATE      CREATED AT
special-fiesta-q75q... special fiesta 23-22411-007-os... main    Available  about 18 minute...
```

 TAKE SCREENSHOT #1 NOW

Screenshot name:

task0_codespace_create_and_list.png

STEP 7: SSH into Codespace (IMPORTANT)

From the same terminal, run:

```
gh codespace ssh -c <codespace_name>
```

Example:

```
gh codespace ssh -c special-fiesta-q75q6qvw7vjpf7gr
```

If successful, your prompt will change to something like:

```
vscode@codespaces-xxxxx:/workspaces/Lab13$
```

```
@23-22411-007-oss →/workspaces/lab13 (main) $ gh codespace ssh -c special-fiesta-q75q6qw7vjpfx7gr
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@23-22411-007-oss →/workspaces/lab13 (main) $
```

 **TAKE SCREENSHOT #2 NOW**

Screenshot name:

task0_codespace_ssh_connected.png

Task 1 — Create IAM Group and Output Details

BEFORE YOU START (VERY IMPORTANT)

Terraform **will NOT work** unless AWS credentials are configured **inside Codespace**.

Check AWS credentials exist

```
@23-22411-007-oss →/workspaces/lab13 (main) $ aws configure
AWS Access Key ID [None]: AKIATFX3NIVRD34V00XA
AWS Secret Access Key [None]: 8CDIJvyrYFIsgyi8lN/9RvX/qA64Ux3bB6cnAACz
Default region name [None]: me-central-1
Default output format [None]: json
@23-22411-007-oss →/workspaces/lab13 (main) $
```

STEP 1: Create Project Directory

Run **exactly**:

mkdir -p ~/Lab13

cd ~/Lab13

pwd

```
@23-22411-007-oss →/workspaces/lab13 (main) $ mkdir -p ~/Lab13
cd ~/Lab13
pwd
/home/codespace/Lab13
@23-22411-007-oss →~/Lab13 $
```

 **TAKE SCREENSHOT**

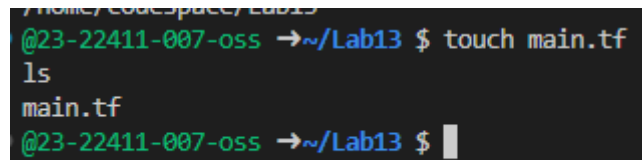
task1_project_directory.png

STEP 2: Create main.tf

Run:

touch main.tf

ls

A terminal window with a dark background. The prompt is '@23-22411-007-oss →~/Lab13 \$'. The user enters 'touch main.tf', followed by 'ls', which shows 'main.tf' in the output. The prompt returns to '@23-22411-007-oss →~/Lab13 \$' with a cursor.

```
@23-22411-007-oss →~/Lab13 $ touch main.tf
ls
main.tf
@23-22411-007-oss →~/Lab13 $
```

TAKE SCREENSHOT

task1_file_created.png

STEP 3: Add Terraform Code to main.tf

Open file:

nano main.tf

PASTE THIS EXACT CODE (no changes):

```
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_iam_group" "developers" {
  name = "developers"
  path = "/groups/"
}

output "group_details" {
  value = {
    group_name = aws_iam_group.developers.name
    group_arn  = aws_iam_group.developers.arn
    unique_id  = aws_iam_group.developers.unique_id
  }
}
```

Save:

- CTRL + O → Enter
- CTRL + X

Verify content:

cat main.tf

```
@23-22411-007-oss →~/Lab13 $ cat main.tf
provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_iam_group" "developers" {
  name = "developers"
  path = "/groups/"
}

output "group_details" {
  value = {
    group_name = aws_iam_group.developers.name
    group_arn  = aws_iam_group.developers.arn
    unique_id  = aws_iam_group.developers.unique_id
  }
}
```

 TAKE SCREENSHOT

task1_main_tf.png

STEP 4: Initialize Terraform

Run:

terraform init

```
@23-22411-007-oss →~/Lab13 $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.30.0...
- Installed hashicorp/aws v6.30.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@23-22411-007-oss →~/Lab13 $ S
```


TAKE SCREENSHOT

task1_terraform_init.png

STEP 5: Apply Terraform Configuration

Run:

terraform apply -auto-approve

```
@23-22411-007-oss →~/Lab13 $ terraform apply -auto-approve
+ unique_id = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ group_details = {
+   group_arn = (known after apply)
+   group_name = "developers"
+   unique_id = (known after apply)
}
aws_iam_group.developers: Creating...
aws_iam_group.developers: Creation complete after 1s [id=developers]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

group_details = {
  "group_arn" = "arn:aws:iam::218496845154:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPATFX3NIVRBEFV6DCL6"
}
@23-22411-007-oss →~/Lab13 $
```

TAKE SCREENSHOT

task1_terraform_apply.png

STEP 6: Display Terraform Output

Run:

terraform output

```
@23-22411-007-oss →~/Lab13 $ terraform output
group_details = {
  "group_arn" = "arn:aws:iam::218496845154:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPATFX3NIVRBEFV6DCL6"
}
@23-22411-007-oss →~/Lab13 $
```

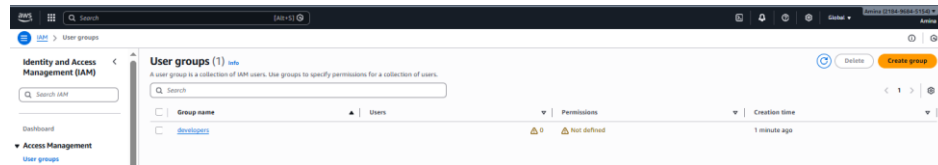
TAKE SCREENSHOT

task1_terraform_output.png

STEP 7: Verify in AWS Console

1. Open **AWS Console**
2. Go to **IAM**

3. Click **Groups**
4. You should see:
5. Developers



📷 TAKE SCREENSHOT

task1_aws_console_group.png

Task 2: Create IAM User with Group Membership

Update main.tf to define an IAM user resource.

```

lab_13 > Task > main.tf > output "user_details"
11  output "group_details" {
17  }
18
19  resource "aws_iam_user" "lb" {
20    name = "loadbalancer"
21    path = "/users/"
22    force_destroy = true
23    tags = {
24      DisplayName = "Load Balancer"
25    }

```

Enable force_destroy to allow clean deletion. Create a user-to-group membership resource. Add Terraform outputs for user details. Apply the updated configuration by terraform apply.

```

~approve
"group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
"group_name" = "developers"
"unique_id" = "AGPA5WPGPNRIWSOVQPNE5"
}
user_details = {
  "unique_id" = "AIDA5WPGPNRI5E5P3HF3E"
  "user_arn" = "arn:aws:iam::941618064465:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}

```

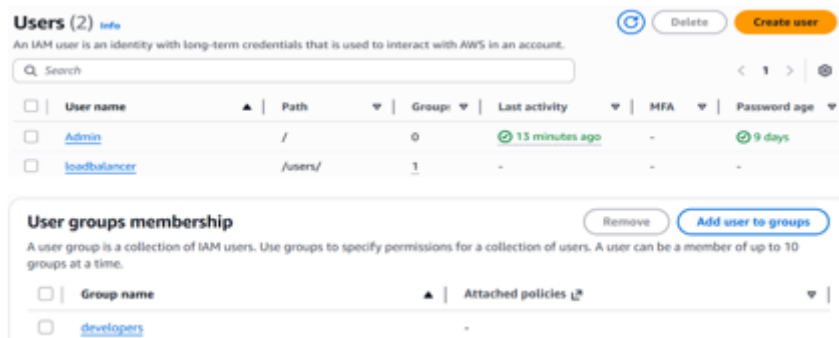
Display Terraform outputs, bybterraform output.

```

group_details = {
  "group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA5WPGPNRIWSOVQPNE5"
}
user_details = {
  "unique_id" = "AIDA5WPGPNRI5E5P3HF3E"
  "user_arn" = "arn:aws:iam::941618064465:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}

```

Verify the user and group membership in the AWS Console.



Task 3: Attach Policies to IAM Group

Update main.tf to add group policy attachment resources.

```
lab_13 > Task > main.tf > resource "aws_iam_group_policy_attachment" "change_password"
35   output "user_details" {
41   }
42
43   resource "aws_iam_group_policy_attachment" "developer_ec2_fullaccess" {
44     group = aws_iam_group.developers.name
45     policy_arn = "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
46   }
```

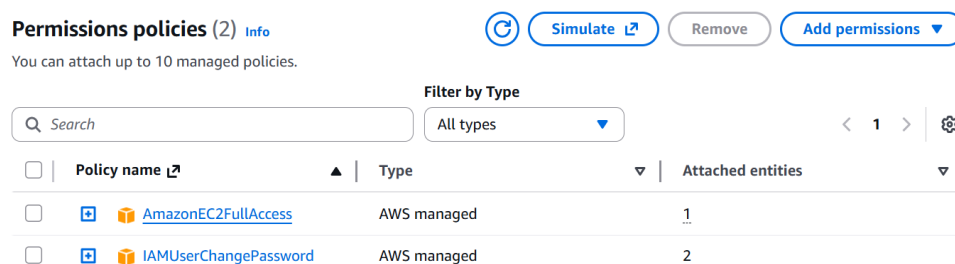
Attach AmazonEC2FullAccess policy. Attach IAMUserChangePassword policy. Apply the Terraform configuration, terraform apply.

```
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

group_details = {
  "group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA5WPGPNRTWSOVQPNF5"
```

Verify attached policies in the AWS Console.



Task 4: Create Login Profile for IAM User

Create variables.tf to store IAM password securely.

```

① README.md U main.tf U variables.tf U X
lab_13 > Task > variables.tf > variable "iam_password"
1  variable "iam_password" {
2      description = "Temporary password for the IAM user"
3      type        = string
4      sensitive   = true
5      default     = "IdontKnow"

```

Create create-login-profile.sh Bash script.

```

① README.md U main.tf U variables.tf U $ create-login-profile.sh U X
lab_13 > Task > $ create-login-profile.sh
1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  USERNAME="$1"
5  PASSWORD="$2"
6
7  # Check if login profile already exists
8  if aws iam get-login-profile --user-name "$USERNAME" >/dev/null 2>&1; then

```

Make the script executable using `chmod +x create-login-profile.sh`. Add a `null_resource` with `local-exec` provisioner in `main.tf`.

```

① README.md U main.tf U X variables.tf U $ create
lab_13 > Task > main.tf > resource "null_resource" "create_login_profile"
28  resource "null_resource" "create_login_profile" {
29      triggers = {
30          password_hash = sha256(var.iam_password)
31          user          = aws_iam_user.lb_name

```

Apply Terraform with a custom password variable, by `terraform apply`.

```

-approve -var="iam_password=MySecurePass123!"
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

group_details = {
  "group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA5WPG6NBTJMSQVQPNF5"

```

Verify login profile using AWS CLI, by `aws iam get-login-profile --user-name loadbalancer`

```

file --user-name loadbalancer
{
  "LoginProfile": {
    "UserName": "loadbalancer",
    "CreateDate": "2026-01-28T08:31:30+00:00",
    "PasswordResetRequired": true
  }
}

```

Test IAM user login in AWS Console.

Old Password

☐ Show Password

New Password

Confirm New Password

☐ Show Password Matches

Confirm Password Change

[Sign in to a different account](#)

Task 5 — Generate Access Keys for IAM User

Add IAM access key resource to main.tf.

```
lab_13 > Task > main.tf > ...
66 # Access key for IAM user
67 resource "aws_iam_access_key" "lb_access_key" {
68     user = aws_iam_user.lb.name
69 }
```

Define outputs for access key ID and secret. Apply Terraform configuration.

```
-approve -var="iam_password=MySecurePass123!"
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Display outputs using terraform output.

```
access_key_id = "AKIA5WPGPNRIYT2DNYFM"
access_key_secret = <sensitive>
group_details = {
  "group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA5WPGPNRIWSOVQPNE5"
}
user_details = {
  "unique_id" = "AIDA5WPGPNRI5E5P3HF3E"
  "user_arn" = "arn:aws:iam::941618064465:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
```

View secret value in terraform.tfstate, cat terraform.tfstate | grep -A 10 "access_key_secret"

```
| grep -A 10 "access_key_secret"
  "access_key_secret": {
    "value": "O98bFIWKW7J5vRngwSzuIVZTiMkUXFtCMoBc9122",
    "type": "string",
    "sensitive": true
  },
  "group_details": {
    "value": {
      "group_arn": "arn:aws:iam::941618064465:group/groups/developers",
      "group_name": "developers",
      "unique_id": "AGPA5WPGPNRIWSOVQPNE5"
    }
  }
}
```

Verify access keys in AWS Console.

Access keys (1)

[Create access key](#)

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

AKIA5WPGPNRIYT2DNYFM

Actions

Description

-

Status

Active

Last used

None

Created

6 minutes ago

Last used region

N/A

Last used service

N/A

Task 6 — Implement Terraform Remote State with S3

Create an S3 bucket with versioning enabled.

Amazon S3 > Buckets > Create bucket

Create bucket

Buckets are containers for data stored in S3.

General configuration

AWS Region
US East (N. Virginia) us-east-1

Bucket type

☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket name
myapp-s3-bucket-l1ab13

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Copy settings from existing bucket - optional

Bucket Versioning

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

☐ Disable

☒ Enable

Add S3 backend configuration to main.tf.

```
lab_13 > Task > main.tf > resource "aws_iam_group_policy"

1 terraform {
2   backend "s3" {
3     bucket = "myapp-s3-bucket-l1ab13"
4     key    = "myapp/terraform.tfstate"
5     region = "us-east-1"
6     encrypt = true
7     use_lockfile = true
8   }
9 }
```

Reinitialize Terraform with state migration.

Terraform has been successfully initialized!

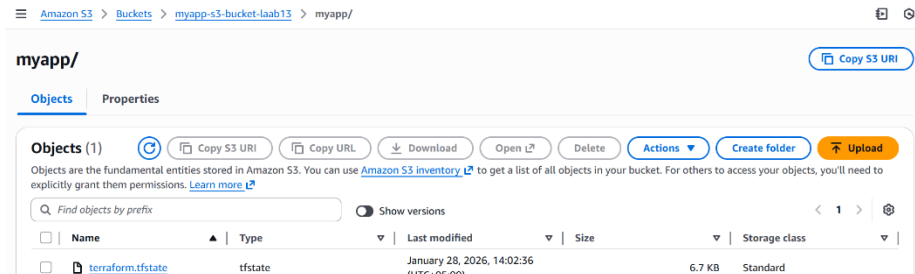
You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

Apply Terraform using remote state.

```
-approve -var="iam_password=MySecurePass123!"  
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
```

Verify state file in S3.

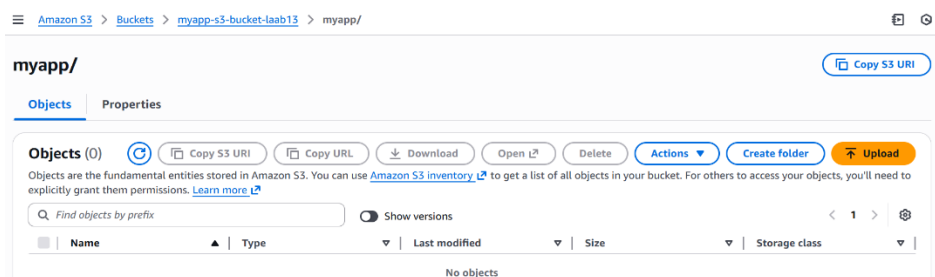


Ls -la terraform.tfstate*

```
-rw-rw-rw- 1 vscode vscode    0 Jan 28 09:02 terraform.tfstate  
-rw-rw-rw- 1 vscode vscode 6882 Jan 28 09:02 terraform.tfstate.backup
```

Destroy resources and verify updated state.

```
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Destruction complete after 1s  
aws_iam_user_group_membership.lb_membership: Destruction complete after 1s  
aws_iam_group.developers: Destroying... [id=developers]  
aws_iam_access_key.lb_access_key: Destruction complete after 1s  
aws_iam_user.lb: Destroying... [id=loadbalancer]  
aws_iam_group.developers: Destruction complete after 1s  
aws_iam_user.lb: Destruction complete after 3s  
  
Destroy complete! Resources: 7 destroyed.
```



Task 7 — Create Multiple Users from CSV File

Create locals.tf to decode CSV data.

```
lab_13 > Task > locals.tf > locals  
1  locals {  
2    users = csvdecode(file("users.csv"))  
3  }
```

Create users.csv with multiple usernames.

```
lab_13 > Task > users.csv
1  user_name
2  Michael
3  Dwight
4  Jim
5  Pam
6  Ryan
7  Andy
```

Update main.tf to create users dynamically.

```
lab_13 > Task > main.tf > resource "aws_iam_access_key" "users_access_keys" > user
30  # Create multiple IAM users from CSV
31  resource "aws_iam_user" "users" {
32    for_each = { for user in local.users : user.user_name => user }
33
34    name      = each.value.user_name
35    path      = "/users/"
```

Assign all users to developers group by terraform init.

```
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

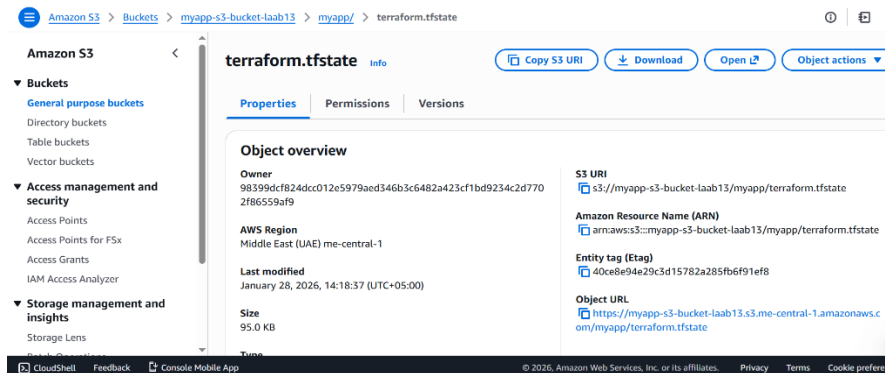
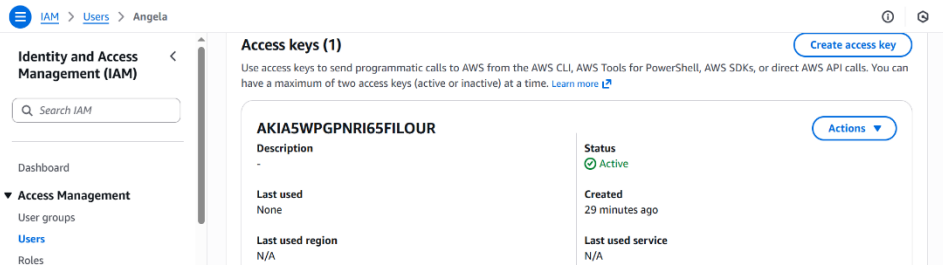
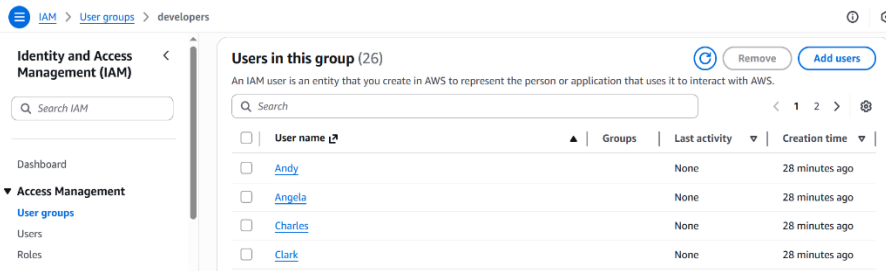
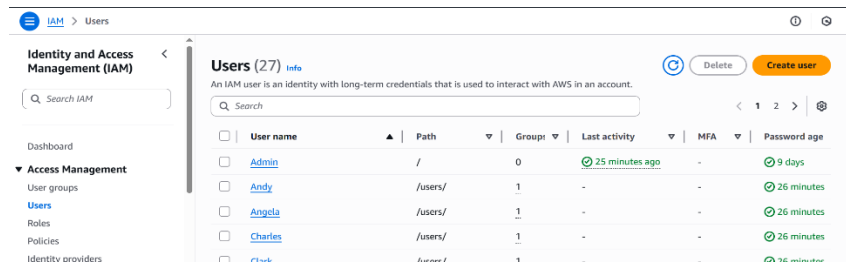
Generate login profiles and access keys. Apply Terraform configuration.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS 2
Apply complete! Resources: 107 added, 0 changed, 0 destroyed.
```

Verify users, groups, and state file, by terraform output.

```
> lab_12
> lab_13 / Task
> terraform
  terraform.lock.hcl  U
$ create-login-profile... U
  locals.tf          U
  main.tf            U
  README.md          U
  terraform.tfstate
  terraform.tfstate.backup
  users.csv          U
  variables.tf       U
> Lab_Exam
> software
> UbuntuMachine
  .gitignore
  MyED25519Key.pem
> OUTLINE
> TIMELINE

"Robert" = {
  "access_key_id" = "AKIA5WPGPNRI7QXNMFBD"
  "user_arn" = "arn:aws:iam::941618064465:user/users/Robert"
  "user_unique_id" = "AIDA5WPGPNRI6MD7NIM48"
}
"Ryan" = {
  "access_key_id" = "AKIA5WPGPNRI4V6ZFA72"
  "user_arn" = "arn:aws:iam::941618064465:user/users/Ryan"
  "user_unique_id" = "AIDA5WPGPNRI2C7VAJ3CC"
}
"Stanley" = {
  "access_key_id" = "AKIA5WPGPNRI2XGVH2"
  "user_arn" = "arn:aws:iam::941618064465:user/users/Stanley"
  "user_unique_id" = "AIDA5WPGPNRI6MAUCS45U"
}
"Toby" = {
  "access_key_id" = "AKIA5WPGPNRI4V6ZFA72"
  "user_arn" = "arn:aws:iam::941618064465:user/users/Toby"
  "user_unique_id" = "AIDA5WPGPNRIQZ5FJFVTT"
}
group_details = {
  "group_arn" = "arn:aws:iam::941618064465:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA5WPGPNRI2D1GHWZXS"
```

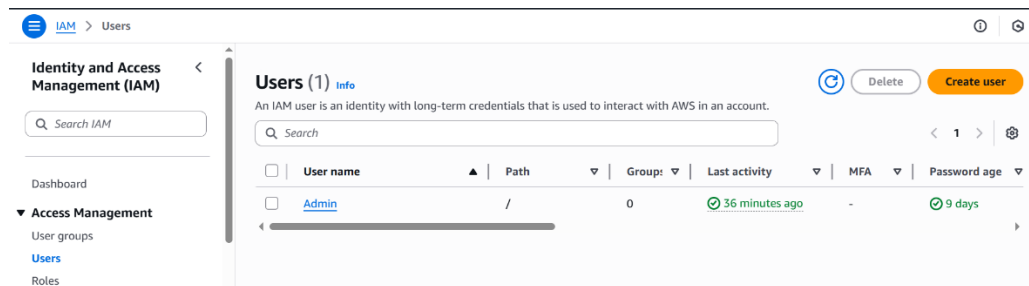
Cleanup — Destroy Resources & Verify State

Destroy all Terraform-managed resources.

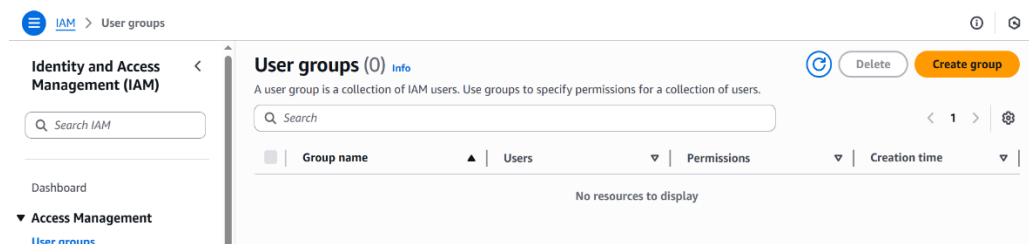
```
aws_iam_user.users["Dwight"]: Destruction complete after 3s
aws_iam_user.users["Phyllis"]: Destruction complete after 3s
aws_iam_user.users["Robert"]: Destruction complete after 3s
aws_iam_user.users["Jim"]: Destruction complete after 5s
aws_iam_user.users["David"]: Destruction complete after 3s
aws_iam_user.users["Peter"]: Destruction complete after 3s
aws_iam_user.users["Michael"]: Destruction complete after 3s
aws_iam_user.users["Meredith"]: Destruction complete after 5s
aws_iam_user.users["Kevin"]: Destruction complete after 8s

Destroy complete! Resources: 107 destroyed.
```

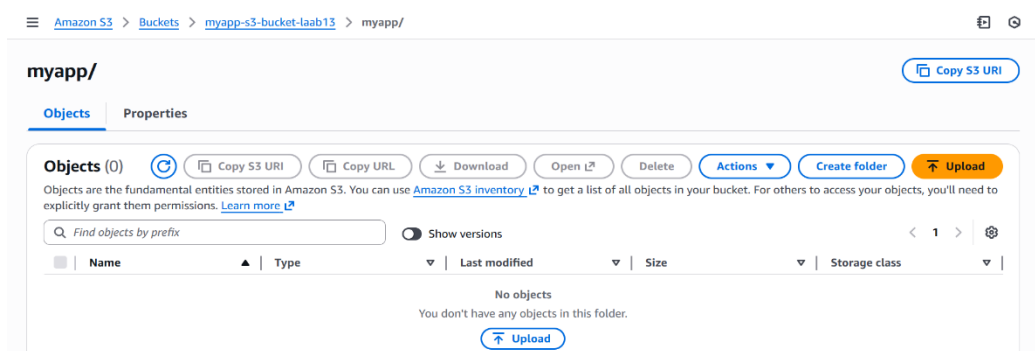
Verify IAM users deletion in AWS Console.



Verify IAM group deletion.



Check S3 state file.



List final project files, by ls -la

```
total 44
drwxrwxrwx+ 3 vscode vscode 4096 Jan 28 09:12 .
drwxrwxrwx+ 3 vscode vscode 4096 Jan 28 07:46 ..
-rwxrwxrwx 1 vscode vscode 422 Jan 28 08:26 create-login-profile.sh
-rw-rw-rw- 1 vscode vscode 49 Jan 28 09:11 locals.tf
-rw-rw-rw- 1 vscode vscode 2535 Jan 28 09:16 main.tf
-rw-rw-rw- 1 vscode vscode 0 Jan 28 07:46 README.md
drwxr-xr-x+ 3 vscode vscode 4096 Jan 28 09:02 .terraform
-rw-r--r-- 1 vscode vscode 2422 Jan 28 08:31 .terraform.lock.hcl
-rw-rw-rw- 1 vscode vscode 0 Jan 28 09:02 terraform.tfstate
-rw-rw-rw- 1 vscode vscode 6882 Jan 28 09:02 terraform.tfstate.backup
-rw-rw-rw- 1 vscode vscode 166 Jan 28 09:13 users.csv
-rw-rw-rw- 1 vscode vscode 149 Jan 28 08:25 variables.tf
```

(Optional) Delete the S3 bucket.

General purpose buckets All AWS Regions Directory buckets

General purpose buckets (0) Info

Copy ARN Empty Delete

Buckets are containers for data stored in S3.

< 1 >

Name	AWS Region	Creation date
------	------------	---------------

No buckets
You don't have any buckets.

Account snapshot Info

Updated daily

Storage Lens provides visibility into storage usage and activity trends.

External access summary Info

Updated daily

External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.